

Optimus-Cost-Agent

Agent Execution Guardrails and Workflow Strategy

Layered Safety Controls and Bounded Execution - Phase 1 Companion Specification

Version 1.2

Architected by: Vibhanshu Agarwal

0. Purpose & Document Scope

This companion specification is the single canonical source for how the Optimus-Cost-Agent constrains what an agent is allowed to *do* at execution time, and how long-running agent work is bounded and made verifiable. It is deliberately maintained outside the core design documents so that the Architecture (HLD) and Low-Level Design (LLD) describe the cost-governance machine without being weighed down by the full guardrail and workflow policy surface. The core documents reference this document; they do not duplicate it.

The material here divides into two complementary halves that together describe the agent's execution contract:

- **Safety controls** — the permission model, the pre-tool guard / hook layer, shell command sanitization, prompt-injection and MCP supply-chain defense, and pre-commit / CI gates. These decide whether a proposed action is permitted to reach execution at all.
- **Execution controls** — bounded goal-driven agent loops and curated workflow skills. These govern how the agent performs large, well-bounded work without context bloat, and how repeatable procedural knowledge is loaded on demand under the same budget and trust discipline.

Both halves are governed by the existing Optimus control plane: the Operating Modes & Trust Framework (HLD §7), Tool Governance & Evidence Acquisition (HLD §8), the Adaptive Agent Execution Strategy & Rigor Policy (HLD §9), the Agent Lifecycle State Machine (LLD §4A), and the Optimus AI Gateway budget and observability ledger (HLD §11 / LLD §10, §10A). Nothing in this document weakens those guarantees; it extends them with explicit enforcement and workflow contracts.

Cross-Reference Register

This document is anchored into the canonical set as follows. The corresponding insert text and version bumps are recorded in the document-control register at the end of this specification (§13).

Document	Anchor section	Relationship
Architecture (HLD)	New §13 — Agent Execution Safety & Guardrails	Short cross-cutting reference; this document holds the detailed policy model.
Low-Level Design (LLD)	New §12 — Guardrail & Workflow Component Contracts	Detailed implementation contract for the Phase 1 enforcement and workflow components (§10 here).
Test Strategy	New §14 — Guardrail & Workflow Test Cases	Source for guardrail / security / loop / skill test categories (§11 here); adds traceability rows.

1. Control-Plane Overview

Permissions and guardrails are the floor, not a nice-to-have. They are the baseline control over what the agent can do without asking a human each time. The premise is not that the agent is malicious — it is that the agent is a *reward-hacky problem solver*. When it is grinding toward "done," its own judgement is the

threat surface. Hit it with a permission error and it reaches for `chmod 777`; a dependency will not install cleanly and it tries `curl ... | sh`; a test keeps failing and it starts commenting out assertions; a `git push` is blocked and it tries to force-push to `main`. Guardrails exist to remove these creative shortcuts from the agent's reach.

Optimus addresses this with a layered control plane. Every proposed tool call passes through deterministic policy and validation before it can mutate anything, executes inside a sandbox, is audited, and — for any change that reaches the repository — is re-checked by commit and CI gates. The same plane wraps bounded execution loops and on-demand skills so that long-running or procedure-driven work inherits the identical constraints. The control flow is shown in Figure 1; each stage maps to a section of this document.

2. Permission Model and Split Authority

Permissions remain allow/deny lists for tool calls, file reads, and shell commands. Mode is evaluated first; user deny rules take precedence over project allows; an ambiguous or high-impact call is held for explicit human approval. A first-time MCP tool, a manifest change, and an effect-class escalation are never silently allowed by a classifier.

2.4 Human approval and MCP authority

The agent-side Plan 6.5 cage remains authoritative for human approval, sanitized descriptor and manifest trust, namespaced `allowed_tools`, `permission_scope`, independently derived effect class, descriptor exposure, and the final pre-tool decision. Approval binds the non-secret manifest hash to the opaque Gateway profile revision; either value changing produces `mcp.manifest_hash_changed` and requires reapproval.

The Gateway independently enforces the operator-provisioned `profile_id`, transport and credential binding, current revision, upstream-name allowlist, resource limits, attribution policy, and lifecycle state. The shared bearer authenticates the caller but does not establish MCP approval. Every MCP call still requires an active profile, exact binding pair, and Gateway allowlist membership, even when an agent-only gate was bypassed.

This is intentionally split authority. A direct shared-secret caller may bypass agent-only scope and effect checks, but cannot exceed the Gateway allowlist or invoke after detected binding or manifest drift. A recoverable refresh failure serves the last approved binding with a stale marker rather than denying it; detected drift remains a denial. This residual is accepted only for the strict-loopback, single-operator Phase 1 deployment and does not claim resistance to a compromised Gateway.

The decision order is therefore: mode check, user deny list, project allow list, impact class, agent-side MCP approval/scope/effect checks, then the Gateway profile/binding/allowlist/freshness/ resource/budget checks. Approval is recorded as an audit event; Gateway admission does not duplicate the agent's permission-scope or effect-class authority.

2.1 Project-Level Allow Rules

Committed to the repository and shared by every contributor and agent, the project layer enumerates the safe, recurring actions that run without prompting: linting, type checks, the test runner, formatters, and read-only git inspection (`status`, `diff`, `log`, `show`). These are the operations whose blast radius is understood and whose cost is negligible.

2.2 User-Level Deny Rules

The user / deployment layer is a deny-list for things that should never happen regardless of context. It always takes precedence over any allow rule and cannot be overridden by a project setting or by the model. The Phase 1 baseline deny set:

- Reads of credential and secret material — `.env`, key files, token stores, cloud credential paths.
- Destructive filesystem operations — `rm -rf`, recursive force deletes, unsafe glob removals outside the workspace root.
- History rewrites and force-push to protected branches — `git push --force` / `--force-with-lease` to `main`.
- Pipe-to-shell execution — `curl ... | sh`, `wget ... | bash`, and equivalents.
- Broad permission changes — `chmod 777` and other world-writable grants.
- Unscoped network egress and environment manipulation (see §4).

2.3 Mode Mapping

The permission model is layered on top of the Operating Modes & Trust Framework (HLD §7) and the Agent Lifecycle State Machine (LLD §4A). Mode is evaluated *before* the allow/deny lists, so a denied mode short-circuits the decision.

Mode	Writes	Shell mutation	External side effects	Allowed
Plan / Chat	No	No	No	Inspect, plan, propose diffs/snippets; append-only internal telemetry.
Agent	Yes*	Yes*	Yes*	Tool calls permitted <i>only</i> through the permission policy and pre-tool guard (§3); mutation only after fitness gates pass.

* In Agent mode every starred capability is still gated by §2.1–§2.2, the pre-tool guard (§3), and the composite fitness engine before any change reaches the working tree.

2.4 Human Approval

Any action that is ambiguous under policy, or whose impact class is high (a `FILE_MUTATION` or `MULTI_FILE_CHANGESET` per HLD §7, a deny-adjacent command, or a first-time MCP/tool), is held for explicit human approval rather than allowed or silently rejected. This is the *Denied / Held for Human Review* branch in Figure 1. Approval is recorded as an audit event (§10) so the decision is replayable.

3. Pre-Tool Guard / Hook Layer

PreToolGuard fires after the agent has assembled a tool call and before execution. Deterministic rules, tables, AST checks, path containment, descriptor scanning, and budget checks run before any borderline classifier. The guard returns ALLOW, BLOCK, or HOLD and writes the same ToolInvocationAuditEvent trail as the permission engine.

3.2 MCP hook surface

For an MCP call the agent gate checks human approval, the bound non-secret manifest hash and Gateway profile revision, namespaced `profile_id`, `tool_name`, descriptor trust, `permission_scope`, and the independently derived effect class. Only the operator-selected descriptor subset enters model context; descriptor count, UTF-8 bytes, and admitted identities are bounded and recorded. Semantic tool search and automatic per-turn selection remain deferred.

The Gateway is a second gate, not a replacement for this cage. It filters upstream tool names against the operator allowlist before descriptors cross, checks profile state, exact binding, freshness, resource limits, and budget, and applies the effective freshness interval `min(local_max_age, ttlMs)`. A failed recoverable refresh marks the last binding `stale_marked`; detected manifest or binding drift closes admission.

3.3 Deferred capability boundary

V1 exposes only the typed tools surface. `input_required` is a typed, call-scoped denial at method, result-type, and content boundaries; it never becomes a planner question, profile-state change, or automatic retry. `resource_link` and embedded resources remain inert untrusted data. Complete-only results are accepted; image/audio blocks are discarded with a typed disposition and never decoded or persisted.

Cost. The guard is pure validation logic — **zero LLM cost** on the deterministic path; the optional classifier is budgeted and rare. Pairing the guard with sandboxing (it is the inspection layer, not a replacement for isolation) keeps the agent moving without constant hand-holding.

4. Shell Command Sanitization

Shell access is the highest-leverage and highest-risk tool surface. Phase 1 defines a local `CommandSafetyValidator` interface invoked by the pre-tool guard (§3) on every Bash call. The validator is deterministic, runs in-process, and blocks before the command reaches a subprocess.

4.1 Required Checks

- **Destructive commands** — recursive force deletes, disk/format operations, and unsafe overwrites outside the workspace root.
- **Pipe-to-shell** — any pattern that fetches and executes in one step (`curl ... | sh`, `... | bash`, process-substitution variants).
- **Credential / environment access** — reads of `.env`, key stores, or attempts to exfiltrate environment variables.
- **Unicode homoglyph / confusable characters** — see §4.2.
- **ANSI / control characters** — terminal-injection sequences embedded in commands or tool output.
- **Insecure transport** — plain-HTTP fetches and downgraded TLS where a secure channel is expected.
- **Unexpected network egress** — outbound connections to hosts outside the gateway-managed allowlist.

4.2 The Homoglyph Problem

The nastiest case is a command that looks completely normal. Two strings can be visually identical yet differ at the code-point level: the Latin "i" is ASCII code point 105, while the Cyrillic "i" is Unicode code point 1110. To the eye they are the same character; to the shell they are different, and the shell runs whatever is actually on the other end. The validator normalizes and inspects code points so a confusable hostname or path cannot smuggle execution past a human reviewer.

```
class CommandSafetyValidator(Protocol):
    def validate(self, command: str, ctx: ToolContext) -> ValidationResult: ...

# ValidationResult.verdict ∈ {ALLOW, BLOCK, HOLD}
# Checks: destructive | pipe_to_shell | env_access | homoglyph
#         | ansi_control | insecure_transport | network_egress
```

4.3 Implementation Posture

Tirith is a validator built for exactly this class of check — hostname and path homoglyphs, insecure transport, ANSI injection, pipe-to-shell patterns, and environment manipulation — and is a credible candidate implementation. The design does *not* depend on Tirith: the contract is `CommandSafetyValidator`, satisfied by **Tirith or an equivalent validator**. Whatever the backing

5. Prompt-Injection & MCP Supply-Chain Defense

Config files, repository instructions, MCP descriptors, tool results, and server annotations are untrusted input. ConfigTrustScanner treats config and rule files as code. MCPTrustRegistry never auto-loads a server from a cloned repository; explicit operator approval, allowed tools, permission scope, manifest identity, and descriptor scanning remain required. A manifest-hash change forces reapproval, and derived effect rules are retained in the agent cage.

5.2 MCP server and Gateway supply-chain controls

The Gateway uses catalog metadata only as an operator-facing pre-provisioning reference. There is no registry trust, agent/model lookup, autoload, install, connect, update, activation, or Gateway data-plane query in v1. Provisioning determines whether a profile may exist; opening, reusing, or closing a transport for an active revision cannot activate or rewrite that profile.

Before descriptors cross the Gateway boundary, the Gateway filters upstream names against the operator allowlist, rejects malformed definitions, preserves descriptor text verbatim, and applies versioned canonicalization. The agent uses `profile_id.tool_name`; the Gateway uses upstream tool names. Remote credential-bearing HTTP uses per-request metadata, `server/discover`, `tools/list`, and `tools/call` at MCP 2026-07-28, with no initialization fallback, client ping, protocol session, or standalone GET stream. Missing version/tools support returns `mcp.protocol_version_unsupported`. Containerized stdio probes discovery first and may negotiate modern or legacy tools-only behavior.

`tools/list` must consume every `nextCursor` page before a manifest is approvable. Page, tool, descriptor-byte, and elapsed-time bounds are enforced. Repeated or malformed cursors and incomplete pages are integrity failures; transient transport failures use the existing `RetryPolicy`; capacity exhaustion is a narrow no-manifest outcome. Effective freshness is `min(local_max_age, ttlMs)`, and a recoverable refresh failure marks the last approved binding stale until detected drift makes reapproval mandatory.

Only valid `x-mcp-header` definitions survive filtering; v1 never emits `Mcp-Param-*`. Arguments are the only agent-originated payload sent upstream. System prompt, conversation history, policy text, approval records, hidden context, and Gateway credentials never cross the boundary. Results remain untrusted: complete-only output is accepted, structured output is validated, resource content is inert, and image/audio content is discarded with a typed note rather than decoded or persisted.

5.3 Deferred features, OAuth, and transport residuals

Elicitation remains closed at the method/result/content triple. A future opening requires an exact capability advertisement, server-attributed rate-limited operator UI, durable `input_required` hold, schema or approved-HTTPS-origin validation, accept/decline/cancel, bounded rounds/deadlines, redaction, and opaque untrusted `requestState`. The original call is not redispached until all conditions exist.

Sampling is absent in v1. A future opening requires a reviewed server-attributed prompt, no inherited system prompt or conversation history, one human decision before budget reservation, a second before returning the model response, normal provider usage, and an `MCPUsageRecord` linked to the initiating profile/tool. These lifecycle and UI changes are owned by P11-FU-13.

OAuth is static-credential-only in v1. Future automatic same-grant refresh is not operator rotation; grant, issuer, resource, subject, scope, client, store, or policy change is rotation and requires reapproval under P11-FU-12. Generalized OWASP reference material belongs in HLD/LLD as explicitly labelled reference guidance; this document extends only normative P11-FEAT-GATEWAY-MCP controls.

Enforced tier: every stdio profile is Docker-contained from an immutable digest image, with no host mounts, devices, or Docker socket; safe `docker run --env NAME (-e NAME)` credential projection; no model, Gateway, other MCP, or telemetry key; bounded duration, output, and read limits; and deterministic termination.

Platform-gated tier: process-count confinement requires real Windows Job Object and Linux/WSL2 evidence before it may be claimed.

Residual tier: Docker daemon trust, image supply-chain trust, and provisioned network egress remain explicit residuals. MCP roots are not a containment boundary. Neither MCP roots nor the deferred model-generated code-execution pattern is a sandbox boundary or a substitute for the enforced stdio controls.

5.3 Explicit Threat Scenarios

Scenario	Vector	Control
Tool metadata poisoning	Malicious instructions hidden in MCP tool name / description / schema	<code>MCPTrustRegistry</code> descriptor inspection + manifest hash
Repo config poisoning	Agent config / rule file in a cloned repo carries embedded instructions	<code>ConfigTrustScanner</code> on ingest; config treated as code
Auto-load supply chain	MCP server bundled in cloned repo loads on open	No auto-load; allowlist + explicit approval

Cost. Registry checks, hashing, and scanning are deterministic — **zero LLM cost**. The defense applies whenever the agent reads input authored outside your team, which is increasingly most of the time.

6. Pre-commit & CI Gates

Local quality gates block bad code before it becomes a commit; CI repeats them where the agent cannot quietly skip them. Together they are the last layer between the agent's output and your git history, and they extend the Testing & Quality Gates posture of HLD §12. What changes with agents is not the mechanism but the value of being slightly distrustful: rules that feel rigid for humans are often exactly right for agents. The agent does not get annoyed or promise to tidy up later — it hits the gate, reads the error, and tries again. The corrective loop is the point; the enforcement teaches.

6.1 Local Pre-commit — Four-Layer Config

- **Standard hygiene hooks** — trailing whitespace, valid YAML/TOML, file-size limit.
- **Ruff** — linting and formatting, with `--fix` to auto-correct what it can.
- **Bandit** — Python security issues such as hardcoded passwords and `eval()`.
- **AST-grep** — structural rules enforced at the syntax-tree level.
- **Secret scanning** — recommended hardening so a hardcoded credential never becomes git history (and an incident) because the model wanted a test to pass quickly.

6.2 CI — Clean-Environment Re-Check

When code is pushed and a PR is opened, a fresh server clones the repository and runs the same checks as pre-commit — lint, type checks, tests, security scans, and structural rules — where they cannot be skipped. This catches local-only enforcement failures: misconfigured hooks, `--no-verify` slips, and broken clean-checkout assumptions. One practical requirement: a **concurrency block that cancels stale runs** when a new push arrives. Agents are prolific branch-pushers; without cancellation, CI minutes burn on runs that were obsolete seconds after they started.

Cost. Pre-commit and CI are **compute cost, not model-token cost**. Concurrency cancellation directly bounds the compute side of agent-driven churn.

7. Bounded Agent Loops / Goal-Driven Execution

An agent loop re-runs a single agent with fresh context each iteration, tracking progress in files, git, task manifests, traces, and the evidence ledger rather than an ever-growing chat context. Every loop has hard `max_iterations`, `max_budget_usd`, `max_wall_clock_minutes`, an explicit completion condition, per-iteration evidence, a clean git-diff check, an active pre-tool guard, human approval for escalation, and stop-on-repeated-failure behavior.

The completion evaluator is a cheap model routed through the strict-loopback Gateway, not the main reasoning model. It uses the developer-owned aggregator account, the same provider-reported USD ledger and OTel/OTLP path as every other model call, and fails closed on missing or malformed usage or cost. `max_budget_usd` is the reviewed USD rename and does not add a cross-run limit.

7.3 MCP indeterminate holds

MCP `tools/call` is never automatically retried because no general idempotency guarantee exists. After timeout or connection loss following dispatch, the outcome is indeterminate. Read-only tools may be re-invoked; a side-effecting (`profile_id`, `tool`) is held by `PreToolGuard` until explicit operator acknowledgment. The hold is durable across agent-session and agent-process restart, and acknowledgment authorizes a new attempt without asserting that the prior attempt failed.

Only `server/discover` and `tools/list` transient failures use the existing capped `RetryPolicy` and restart a complete scan. Cursor-integrity, authorization, schema, policy, accounting, and other safety failures do not retry automatically. Typed dispositions identify retryability and operator action without credentials, raw authorization challenges, or unredacted server text.

The completion evaluator uses the developer-owned aggregator account through the Gateway, has no direct provider credential or provider adapter, and emits OTel/OTLP telemetry through authenticated Gateway trace ingress with no separate observability backend or billing path.

8. Curated Workflow Skills

Project configuration is for always-on rules; skills are on-demand procedural workflows loaded only when relevant. A skill is Markdown with YAML frontmatter, a focused procedure, applicable globs, allowed tools, owner, version, and trust level. Generated skills are draft-only until reviewed.

Skills inherit the same permission posture. Declared `allowed_tools` are enforced by the pre-tool guard; a skill cannot widen the agent tool surface or override project/user deny rules. The `SkillRegistry` resolves a matching manifest only when description/globs match, and `SkillTrustPolicy` blocks untrusted or draft skills in Agent mode.

8.3 MCP capability and connection posture

The MCP capability table is tools-only in v1: no roots, sampling, elicitation, logging, subscriptions, completion, task streams, arbitrary methods, or automatic resource dereference. Remote HTTP is request-scoped at 2026-07-28; stdio is a bounded Docker child with discovery-first modern/legacy negotiation. Provisioning state and connection lifetime are separate axes; a socket or child cannot create, activate, re-enable, or change a profile.

9. Cost Model Alignment

The governing rule remains rules first, a small-model classifier only when needed, and human approval for high-risk uncertainty. Permission rules, the pre-tool guard, shell validation, MCP defense, and the deterministic MCP discovery/result gates are zero-LLM-cost controls. Pre-commit/CI is compute, not token, cost; bounded loops use a cheap evaluator and hard USD budgets.

Every model-touching element is routed through the same loopback Gateway, developer-owned aggregator account, USD budget, provider-reported cost ledger, and OTel/OTLP trace path. Guardrails introduce no second credential, direct provider adapter, ungoverned cost path, or observability backend dependency.

9.1 MCP accounting states

Existing settled GatewayUsage and ProviderUsage rows remain unchanged. MCP uses a separate MCPUsageRecord keyed by gateway_request_id, profile/revision, namespaced and upstream tool names, transport, disposition, resource and byte fields, duration, and attribution_state.

attribution_state is exactly settled, explicit_zero, or unavailable. settled requires authoritative billing units and cost_usd; explicit_zero requires a revision-bound operator declaration of free external charge; unavailable has absent monetary fields and is never displayed or reconciled as zero. Strict-dollar policy denies unavailable attribution with mcp.budget.unattributed_spend_denied unless the revision-bound policy explicitly permits it.

If accounting persistence fails after execution, the result is withheld and the run is held until the same gateway_request_id is persisted. Recovery retries persistence only and never redispaches the upstream tool call.

10. Implementation Contracts (LLD Anchor)

The authoritative components are PreToolGuard, ToolInvocationAuditEvent, MCPTrustRegistry, ConfigTrustScanner, Gateway profile and discovery/call brokers, the result validator, and the MCP-specific usage writer. The Guardrails layer owns policy and acknowledgment behavior; detailed route, profile, transport, result, and accounting shapes remain in LLD §12.

10.1 Component Inventory

Domain	Contracts
Permissions (§2)	PermissionPolicy, PermissionDecision
Pre-tool guard (§3)	PreToolGuard, ToolInvocationAuditEvent
Shell (§4)	CommandSafetyValidator
Injection / MCP (§5)	MCPTrustRegistry, ConfigTrustScanner
Bounded loops (§7)	GoalLoopController, IterationState, CompletionEvaluator, ProgressLedger, LoopBudgetPolicy, LoopStopReason
Skills (§8)	SkillRegistry, SkillManifest, SkillTrustPolicy, SkillInvocationPolicy

10.2 Representative Contract Shapes

Sketched in the LLD's pydantic idiom for continuity; field sets are finalized in LLD §12.

11. Test Coverage Mapping (Test Strategy Anchor)

Every control in this document must trace to an executable test category. The detailed evidence belongs in Test Strategy §14; these rows define the Guardrails-to-Test-Strategy seam.

Control	Contract / owner	Required evidence category
Permission order and split agency	PreToolGuard plus Gateway profile/allowlist; P11 - FEAT - GATEWAY - MCP	Agent approval/scope/effect and direct-bearer Gateway allowlist tests
No-autoload and descriptor trust	MCPTrustRegistry, ConfigTrustScanner	Cloned-repository autoload denial, manifest reapproval, exact descriptor scan
Namespace and pagination	profile_id.tool_name; complete nextCursor scan	Collision, cursor-integrity, transient-retry, capacity, and freshness tests
Result and deferred-feature boundaries	complete-only, inert resources, call-scoped input_required	Result/schema/content denial; image/audio discard; no profile mutation
Stdio containment	Docker digest, no mounts/devices/socket, --env NAME	Windows/Linux process evidence, timeout/output/read bounds, termination
Accounting and indeterminate holds	MCPUsageRecord; durable acknowledgment	three attribution states, strict unknown-cost denial, persistence-only recovery
Retry and connection separation	existing RetryPolicy; profile versus transport lifecycle	no automatic tools/call retry; discovery retry cap; open/close cannot activate

11.2 Required MCP test cases

- Direct shared-secret calls cannot exceed the Gateway allowlist; agent-only scope/effect checks remain distinct and the strict-loopback residual is documented.
- A descriptor or manifest change forces reapproval; exact upstream descriptor text, canonicalization, namespace, and admitted count/bytes are verified.
- Remote HTTP uses 2026-07-28 per-request metadata and the exact tools-only method set; stdio proves discovery-first modern/legacy negotiation inside Docker.
- Pagination is complete or absent; repeated/malformed cursors reject discovery without a prefix; transient failures retry and capacity exhaustion yields no manifest.
- Results are complete-only; resources are inert; invalid x-mcp-header is excluded; no Mcp-Param-* is emitted; image/audio is discarded; input_required is call-scoped.
- settled, explicit_zero, and unavailable accounting preserve never-zero-for-unknown behavior; persistence failure holds the run without redispach.
- Side-effecting indeterminate calls require acknowledgment and the durable hold survives restart; read-only retry remains allowed.

Generalized OWASP reference rows are not Guardrails acceptance criteria. Only explicitly labelled NORMATIVE — P11 - FEAT - GATEWAY - MCP controls create these MCP test obligations.

11.1 Guardrail & Workflow Traceability

Control	Contract	Test category
Permission decision order (§2)	<code>PermissionPolicy</code>	Permission policy tests
Shell sanitization (§4)	<code>CommandSafetyValidator</code>	Shell validator tests
Homoglyph / confusable (§4.2)	<code>CommandSafetyValidator</code>	Unicode / homoglyph tests
Injection defense (§5)	<code>ConfigTrustScanner</code>	Prompt-injection fixture tests
MCP auto-load denial (§5.2)	<code>MCPTrustRegistry</code>	MCP autoload denial tests
Local vs CI parity (§6)	pre-commit + CI config	Pre-commit / CI parity tests
Bounded loop stop conditions (§7)	<code>GoalLoopController</code>	Loop control tests
Skill match & trust (§8)	<code>SkillRegistry</code> / <code>SkillTrustPolicy</code>	Skill loading & trust tests

11.2 Required Test Cases

- **Permission** — deny precedence over allow; mode short-circuit; impact-class hold; classifier cannot overturn a deny.
- **Shell validator** — destructive, pipe-to-shell, env-access, ANSI, insecure-transport, and egress patterns all BLOCK before subprocess spawn.
- **Unicode / homoglyph** — Cyrillic-vs-Latin confusables in host and path are detected and held/blocked.
- **Prompt-injection fixtures** — poisoned config and poisoned tool metadata are caught on ingest.
- **MCP** — server bundled in a cloned repo does not auto-load; manifest-hash change forces re-approval; allowed-tools enforced.
- **Pre-commit / CI parity** — the same rule set fails identically locally and in a clean CI checkout.
- **Bypass tests** — `--no-verify`, force-push to `main`, unsafe `.env` reads, and unsafe network commands are all blocked.
- **Loop control** — stops on completion, on `max_iterations`, on budget exhaustion, and on repeated-failure detection; never bypasses §2/§3.
- **Skills** — a skill loads only on match; an untrusted/draft skill is blocked; declared allowed-tools are enforced; a skill cannot override project/user deny rules.

12. References

External sources consulted for alignment of the controls above: Claude Code's hook lifecycle and the `PreToolUse` model; the OWASP LLM Top 10 risk catalogue; pre-commit behavior and Ruff's pre-commit

13. Document Control & Cross-Reference Register

This companion document is referenced from the canonical set. The following entries preserve the historical chain and identify the actual changed-page scope; the final publication manifest remains the authority after layout verification.

Document	Was	Becomes	Changed-page scope / insert
Architecture (HLD)	v2.14	v2.15	New §13 safety/guardrails cross-cutting section and Figure 1 reference.
Low-Level Design	v2.37	v2.38	New §12 guardrail/workflow component contracts.
Test Strategy	v1.3	v1.4	New §14 guardrail/workflow test cases and traceability rows.
This document	—	v1.0	Initial consolidated agent-execution safety/workflow issue.
This document	v1.0	v1.1	Local-Gateway USD field/cost-path correction: cover plus changed pages 10-12; no cross-run limit.
This document	v1.1	v1.2	MCP amendment: changed pages 1, 4, 6, 8, 10-12, 14, and 16; final map follows rendered fragment evidence.

The local-Gateway v3 source explicitly reaffirmed the no-MCP disposition as global rule 14 while moving to the loopback architecture. It recorded no causal rationale for that disposition. The hosted-SaaS-premise explanation is therefore unconfirmed, not a fact silently carried into v1.2. This amendment adds the approved typed Gateway MCP surface; it does not rewrite that provenance.

No generalized OWASP reference is made normative here. Normative MCP controls are separately owned by P11-FEAT-GATEWAY-MCP; deferred OAuth, elicitation, sampling, and reverse research-to-document freshness work retain their named custody entries.